

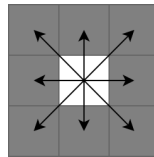
Problem A

Easy as ABC

You are playing a word puzzle. The puzzle starts with a 3 by 3 grid, where each cell contains either the letter A, B, or C.

The goal of this puzzle is to find the lexicographically smallest possible word of length 3. The word can be formed by choosing three different cells where the cell containing the first letter is adjacent to the cell containing the second letter, and the cell containing the second letter is adjacent to the cell containing the third letter.

Two cells are adjacent to each other if they share a border or a corner, as shown in the following illustration. Formally, if (r, c) denotes the cell in the r^{th} row and c^{th} column, then cell (r, c) is adjacent to cell $(r, c + 1)$, $(r - 1, c + 1)$, $(r - 1, c)$, $(r - 1, c - 1)$, $(r, c - 1)$, $(r + 1, c - 1)$, $(r + 1, c)$, and $(r + 1, c + 1)$.



Determine the lexicographically smallest possible word of length 3 that you can find within the grid.

A string s of length n is lexicographically smaller than string t of the same length if there exists an integer $1 \leq i \leq n$ such that $s_j = t_j$ for all $1 \leq j < i$, and $s_i < t_i$ in alphabetical order. The following illustration shows some examples on some grids and their the lexicographically smallest possible word of length 3 that you can find within the grids.

<table><tr><td>B</td><td>C</td><td>B</td></tr><tr><td>C</td><td>A</td><td>C</td></tr><tr><td>B</td><td>C</td><td>B</td></tr></table>	B	C	B	C	A	C	B	C	B	<table><tr><td>B</td><td>C</td><td>B</td></tr><tr><td>C</td><td>C</td><td>C</td></tr><tr><td>C</td><td>C</td><td>A</td></tr></table>	B	C	B	C	C	C	C	C	A	<table><tr><td>A</td><td>C</td><td>A</td></tr><tr><td>C</td><td>B</td><td>C</td></tr><tr><td>A</td><td>C</td><td>A</td></tr></table>	A	C	A	C	B	C	A	C	A	<table><tr><td>A</td><td>C</td><td>A</td></tr><tr><td>C</td><td>A</td><td>C</td></tr><tr><td>A</td><td>C</td><td>A</td></tr></table>	A	C	A	C	A	C	A	C	A	<table><tr><td>C</td><td>C</td><td>C</td></tr><tr><td>C</td><td>B</td><td>C</td></tr><tr><td>C</td><td>C</td><td>C</td></tr></table>	C	C	C	C	B	C	C	C	C
B	C	B																																															
C	A	C																																															
B	C	B																																															
B	C	B																																															
C	C	C																																															
C	C	A																																															
A	C	A																																															
C	B	C																																															
A	C	A																																															
A	C	A																																															
C	A	C																																															
A	C	A																																															
C	C	C																																															
C	B	C																																															
C	C	C																																															

Input

Input consists of three lines, each containing three letters, representing the puzzle grid. Each letter in the grid can only be either A, B, or C.

Output

Output the lexicographically smallest possible word of length 3 that you can find within the grid.

Sample Input #1

```
BCB
CAC
BCB
```

Sample Output #1

ABC

Sample Input #2

BCB
CCC
CCA

Sample Output #2

ACB

Sample Input #3

ACA
CBC
ACA

Sample Output #3

ABA

Sample Input #4

ACA
CAC
ACA

Sample Output #4

AAA

Sample Input #5

CCC
CBC
CCC

Sample Output #5

BCC

Problem B

Button Pressing

You are given N buttons (numbered from 1 to N) and N lamps (numbered from 1 to N). Each lamp can either be on or off. Initially, lamp i is on if $A_i = 1$, and off if $A_i = 0$.

Button i is connected to lamp $i - 1$ (if $i > 1$) and lamp $i + 1$ (if $i < N$). In one move, you can press a button i **only if lamp i is on**. When a button is pressed, the state of the lamps connected to this button is toggled. Formally, the lamps will be on if it was off previously, and the lamps will be off if it was on previously. Note that lamp i is not connected to button i , thus, the state of lamp i does not change if button i is pressed.

After zero or more moves, you want lamp i to be on if $B_i = 1$, and off if $B_i = 0$. Determine if it is possible to achieve this task.

Input

This problem has multiple test cases. The first line consists of an integer T ($1 \leq T \leq 1000$), which represents the number of test cases. Each test case consists of three lines.

The first line of each test case consists of an integer N ($3 \leq N \leq 200\,000$). The sum of N over all test cases does not exceed 200 000.

The second line of each test case consists of a string A of length N . Each character of A can either be 0 or 1. The i^{th} character represents the initial state of lamp i .

The third line of each test case consists of a string B of length N . Each character of B can either be 0 or 1. The i^{th} character represents the desired final state of lamp i .

Output

For each test case, output YES in a single line if the final state of all lamps can be reached after zero or more moves, or NO otherwise.

Sample Input #1

```
2
4
0101
0100
3
000
010
```

Sample Output #1

```
YES
NO
```

Explanation for the sample input/output #1

For the first test case, by pressing the buttons 4, 2, 4, 3, 1, 2 in sequence, the condition of the buttons changes as: 0101 $\rightarrow$ 0111 $\rightarrow$ 1101 $\rightarrow$ 1111 $\rightarrow$ 1010 $\rightarrow$ 1110 $\rightarrow$ 0100.

For the second test case, you are unable to press any button, hence it is impossible to reach the final state.

Sample Input #2

```
5
7
0101011
1111010
5
11111
00000
4
1101
1101
6
101010
100100
3
000
000
```

Sample Output #2

```
NO
NO
YES
YES
YES
```

Problem C

Cursed Game

You found an antique box in your warehouse and decided to open it. The moment you open the box, it traps you into a cursed game where you are playing against a demon. The game consists of 333 rounds and you have to win all rounds in order to escape. The demon also gives you 999 coins that you can use throughout all rounds.

Note that in this problem, denote cell (r, c) of a grid as the cell in row r and column c of the grid.

Before the start of each round, the demon will prepare a *secret paper*, which can be represented as a grid with 3 rows and 3 columns, both numbered from 1 to 3. The demon will secretly put a hole in **one or more cells**, and you are not aware which cells have a hole in it. Then, the round starts with the demon giving you an **odd** integer N ($3 \leq N \leq 33$).

Within each round, you can ask the demon several queries, each costing you one coin. For each query, you are allowed to give the demon *your paper*, which can be represented as a grid with N rows and N columns, both numbered from 1 to N . Each cell is coloured either black or white by you.

For each of your query, the demon will calculate a binary *result grid* with $N - 2$ rows and $N - 2$ columns, both numbered from 1 to $N - 2$. The value on cell (r, c) of the result grid is filled as follows.

- The demon will put the secret paper on top of your paper such that the cell $(r + i - 1, c + j - 1)$ of your paper aligns with the cell (i, j) of the secret paper, for $1 \leq i, j \leq 3$.
- The demon can only see the colour of the cell in your paper if the corresponding cell in the secret paper has a hole in it.
- The value on cell (r, c) of the result grid is 1 if there is an odd number of black cells that it can see through the holes, or 0 otherwise.

You win the round if the result grid consists of only the values 1. Otherwise, the demon will give you the result grid as a feedback, and the round continues.

If you have spent all the coins and still did not win all the rounds, then you will be trapped forever. Escape the cursed game!

Interaction

Each round is started with an odd integer N ($3 \leq N \leq 33$) which can be read through standard input.

Then, for each query that you give to the demon, you can output N lines to standard output. Each of the N lines consists of N characters. The c^{th} character of the r^{th} line represents the colour of cell (r, c) in your paper. The character should be 1 if (r, c) is coloured black, or 0 if coloured white.

The demon will reply with one line consisting of a string, which can be read through standard input.

- If the string is **CORRECT**, then you win the current round and the next round (if exists) will immediately start.
- If the string is **INCORRECT**, then the demon will give you $N - 2$ more lines which can be read through standard input. Each of the $N - 2$ lines consists of $N - 2$ characters, representing the binary result grid as explained in the description.

The demon prepares the secret paper before the start of each round. In other words, **the grader is NOT adaptive**. There will be at least one hole in the secret paper.

The total number of queries across all 333 rounds should not exceed 999. If you exceed the maximum number of queries, you should terminate your program with 0 to receive the verdict *Wrong Answer*. If you do not terminate, the judging result would be undefined because your program is reading from a closed stream.

Do not forget to flush output buffers after each output. In C you can use `fflush(stdout)`. In C++ you can use `fflush(stdout)` or `cout << flush`. In Java you can use method `flush` for output stream, for example, `System.out.flush()`. In Python you can use `stdout.flush()`.

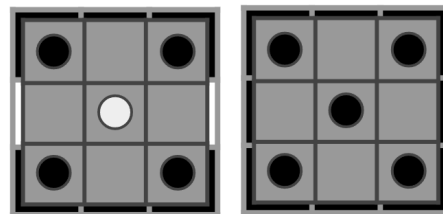
Sample Interaction #1

The following interaction only shows 2 rounds. The actual interaction stays until you win all 333 rounds or you run out of coins.

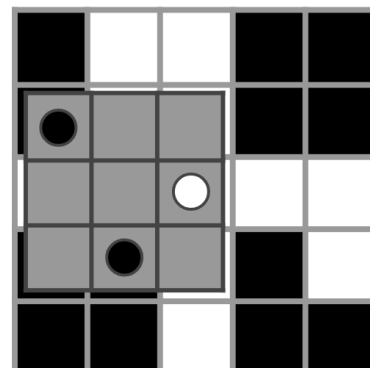
Standard Input	Standard Output
3	
	111 000 111
INCORRECT 0	
	111 111 111
CORRECT	
5	
	10011 10011 00000 11010 11011
INCORRECT 111 001 111	
	11011 11011 00100 11011 11011
CORRECT	
(start of round 3, continue until 333 rounds)	

Explanation for the sample interaction #1

For the first round, the following illustration shows how the demon finds the value in cell (1, 1) of the result grid for the first and second queries. The grey-colored square represents the secret paper, and the circles represent the holes. In the first query, there are 4 black cells that can be seen through the holes, therefore the value in cell (1, 1) of the result grid is 0. In the second query, there are 5 black cells that can be seen through the holes, therefore the value in cell (1, 1) of the result grid is 1. As the result grid consists of only 1, the first round ends.



For the second round, the following illustration shows how the demon find the value in cell (2, 1) of the result grid for the first query. Since there are 2 black cells that can be seen through the holes, the value in cell (2, 1) is 0.



Problem D

Spaceship Exploration

In The ICPC Galaxy, there exists a zone filled with asteroids that is unsafe to enter. The map of the galaxy is represented in a 2D Cartesian coordinate system. The zone is in the shape of an N -sided convex polygon. Each corner is numbered from 1 to N ; corner i is located at (X_i, Y_i) . At any moment, you should not be inside this polygon; however, it is safe to touch the side of the polygon.

There are Q scenarios (numbered from 1 to Q). In scenario j , you want to go from a starting point at (A_j, B_j) to an ending point at (C_j, D_j) . You will be riding on a special spaceship that can only travel in a straight line. First, you set the direction of the spaceship, then the spaceship will start traveling in that direction. During the travel, you are only allowed to change direction **at most once**. Changing direction means you stop the spaceship, set a new direction, and then start traveling again in the new direction.

For each scenario, determine the minimum distance required to travel without being inside of the zone at any moment, or report if it is impossible to reach the ending point.

Input

The first line consists of an integer N ($3 \leq N \leq 100\,000$).

Each of the next N lines consists of two integers X_i, Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$). The points form a convex polygon in counterclockwise order. There are no three points which are collinear.

The following line consists of an integer Q ($1 \leq Q \leq 100\,000$).

Each of the next Q lines consists of four integers A_j, B_j, C_j, D_j ($-10^9 \leq A_j, B_j, C_j, D_j \leq 10^9$). There are no starting points and ending points inside the zone. However, it is possible for the starting point and the ending point to be at the side of the zone.

All the coordinates in the input are integers.

Output

For each scenario, output the answer in a single line.

If it is possible to reach the ending point without being inside the zone at any moment, then output the minimum distance required to travel. Otherwise, output -1 .

Your answer is considered correct if its absolute error or relative error does not exceed 10^{-6} . Namely, if your answer is a and the jury's answer is b , then your answer is accepted if $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$.

Sample Input #1

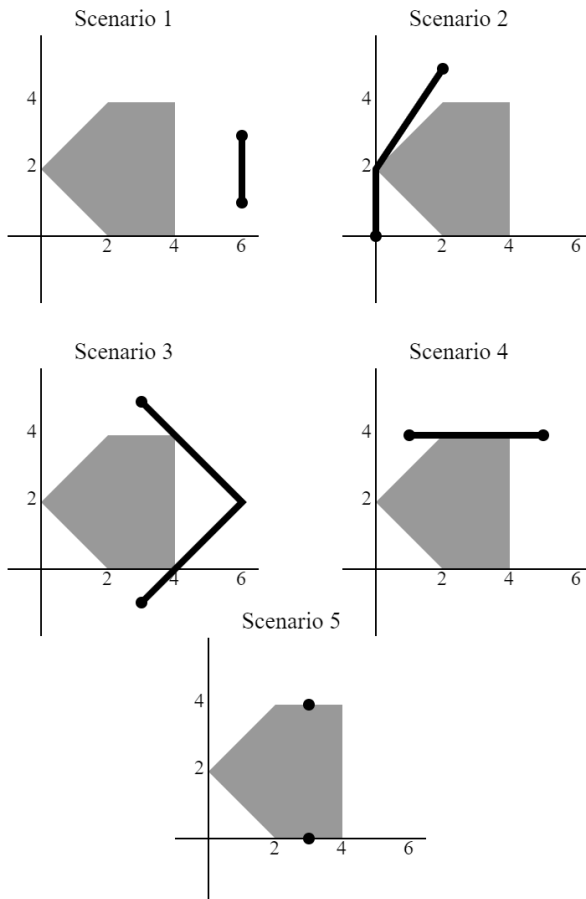
```
5
0 2
2 0
4 0
4 4
2 4
5
6 1 6 3
2 5 0 0
3 5 3 -1
1 4 5 4
3 4 3 0
```

Sample Output #1

```
2
5.6055512755
8.48528137422
4
-1
```

Explanation for the sample input/output #1

This sample is depicted in the following illustration.



During scenario 1 and 4, you can directly go to the ending point without changing the direction.

During scenario 2, you can go to (0, 2), then change direction to the ending point.

During scenario 3, you can go to (6, 2), then change direction to the ending point.

During scenario 5, it can be shown that it is impossible to reach the ending point.

Sample Input #2

```
4
-10 -9
10 -9
10 9
-10 9
2
0 10 0 -10
-10 -10 -10 -10
```

Sample Output #2

```
200.9975124224
0
```

Sample Input #3

```
8
-20 -10
10 -20
25 -15
35 -5
30 10
15 20
-25 15
-30 5
6
-15 -15 -15 20
-30 -5 30 15
25 20 -5 -20
-5 25 20 -20
-30 10 30 -10
-30 -50 50 0
```

Sample Output #3

```
59.0857761929
103.2455532034
94.7213595500
101.5640991922
164.8528137424
94.3398113206
```


Problem E

Merge Not Sort

You are currently researching the Merge Sort algorithm. Merge Sort is a sorting algorithm that is based on the principle of Divide and Conquer. It works by dividing an array into two subarrays of equal length, sorting each subarray, then merging the sorted subarrays back together to form the final sorted array.

You are particularly interested in the merging routine. Common merge implementation will combine two subarrays by iteratively comparing their first elements, and move the smaller one to a new merged array. More precisely, the merge algorithm can be presented by the following pseudocode.

```
Merge( $A[1..N]$ ,  $B[1..N]$ ):  
     $C = []$   
     $i = 1$   
     $j = 1$   
    while  $i \leq N$  AND  $j \leq N$ :  
        if  $A[i] < B[j]$ :  
            append  $A[i]$  to  $C$   
             $i = i + 1$   
        else:  
            append  $B[j]$  to  $C$   
             $j = j + 1$   
    while  $i \leq N$ :  
        append  $A[i]$  to  $C$   
         $i = i + 1$   
    while  $j \leq N$ :  
        append  $B[j]$  to  $C$   
         $j = j + 1$   
    return  $C$ 
```

During your research, you are keen to understand the behaviour of the merge algorithm when arrays A and B are not necessarily sorted. For example, if $A = [3, 1, 6]$ and $B = [4, 5, 2]$, then $\text{Merge}(A, B) = [3, 1, 4, 5, 2, 6]$.

To further increase the understanding of the merge algorithm, you decided to work on the following problem. You are given an array C of length $2 \cdot N$ such that it is a permutation of 1 to $2 \cdot N$. Construct any two arrays A and B of the **same length** N , such that $\text{Merge}(A, B) = C$, or determine if it is impossible to do so.

Input

The first line consists of an integer N ($1 \leq N \leq 1000$).

The following line consists of $2 \cdot N$ integers C_i . The array C is a permutation of 1 to $2 \cdot N$.

Output

If it is impossible to construct two arrays A and B of length N such that $\text{Merge}(A, B) = C$, then output -1 .

Otherwise, output the arrays A and B in two lines. The first line consists of N integers A_i . The second line consists of N integers B_i . If there are several possible answers, output any of them.

Sample Input #1

```
3
3 1 4 5 2 6
```

Sample Output #1

```
3 1 6
4 5 2
```

Explanation for the sample input/output #1

The solution $A = [3, 1, 4]$ and $B = [5, 2, 6]$ is also correct.

Sample Input #2

```
4
1 2 3 4 5 6 7 8
```

Sample Output #2

```
2 3 5 7
1 4 6 8
```

Explanation for the sample input/output #2

The solution $A = [1, 2, 3, 4]$ and $B = [5, 6, 7, 8]$ is also correct.

Sample Input #3

```
2
4 3 2 1
```

Sample Output #3

```
-1
```

Problem F

Maximize The Value

You are given a one-based array consisting of N integers: $A_1, A_2, \dots, A_N$. Initially, the value of each element is set to 0.

There are M operations (numbered from 1 to M). Operation i is represented by $\langle L_i, R_i, X_i \rangle$. If operation i is executed, all elements A_j for $L_i \leq j \leq R_i$ will be increased by X_i .

You have to answer Q **independent** queries. Each query is represented by $\langle K, S, T \rangle$ which represents the following task. Choose a range $[l, r]$ satisfying $S \leq l \leq r \leq T$, and execute operations $l, l+1, \dots, r$. The answer to the query is the maximum value of A_K after the operations are executed among all possible choices of l and r .

Input

The first line consists of two integers N M ($1 \leq N, M \leq 100\,000$).

Each of the next M lines consists of three integers L_i R_i X_i ($1 \leq L_i \leq R_i \leq N$; $-100\,000 \leq X_i \leq 100\,000$).

The following line consists of an integer Q ($1 \leq Q \leq 100\,000$).

Each of the next Q lines consists of three integers K S T ($1 \leq K \leq N$; $1 \leq S \leq T \leq M$).

Output

For each query, output in a single line, an integer which represent the answer of the query.

Sample Input #1

```
2 6
1 1 -50
1 2 -20
2 2 -30
1 1 60
1 2 40
2 2 10
5
1 1 6
2 1 6
1 1 3
2 1 3
1 1 2
```

Sample Output #1

```
100
50
0
0
-20
```

Explanation for the sample input/output #1

For query 1, one of the solutions is to execute operation 4 and 5.

For query 2, one of the solutions is to execute operation 4, 5, and 6.

For query 3, the only solution is to execute operation 3.

For query 4, the only solution is to execute operation 1.

For query 6, the only solution is to execute operation 2.

Sample Input #2

```
5 3
1 3 3
2 4 -2
3 5 3
6
1 1 3
2 1 3
3 1 3
3 2 3
2 2 3
2 2 2
```

Sample Output #2

```
3
3
4
3
0
-2
```

Problem G

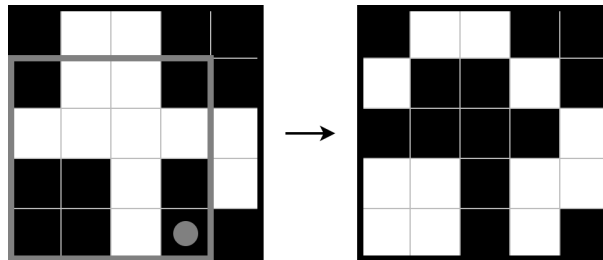
Grid Game 2

You are playing “Grid Game 2” with your friend. There is a grid with 10^9 rows (numbered from 1 to 10^9) and 10^9 columns (numbered from 1 to 10^9). The cell at row r and column c is denoted as (r, c) .

Each cell can have a colour of either black or white. Initially, there are exactly N black cells (numbered from 1 to N). Black cell i is located at (R_i, C_i) . The rest of the cells are white.

You and your friend will alternately take turn playing on this grid, and you are playing in the first turn. In one turn, a player will choose a black cell (r, c) , then toggle cells $(r - x, c - y)$ for all $0 \leq x, y < \min(r, c)$. If a cell is toggled, then the cell becomes black if it was a white cell, and the cell becomes white if it was a black cell.

For example, the following illustration shows how the grid changes after a player chooses a black cell $(5, 4)$ in their turn.



A player who is unable to play on their turn, i.e. no remaining black cells, loses the game, and the opposing player wins the game. If you and your friend are playing optimally, determine who will win the game.

Input

The first line consists of an integer N ($1 \leq N \leq 200\,000$).

Each of the next N lines consists of two integers $R_i C_i$ ($1 \leq R_i, C_i \leq 10^9$). For $1 \leq i < j \leq N$, $(R_i, C_i) \neq (R_j, C_j)$.

Output

Output FIRST if you will win the game, or SECOND otherwise.

Sample Input #1

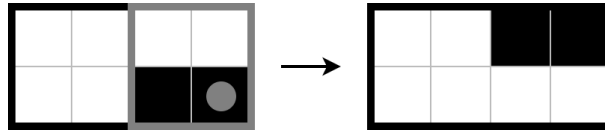
```
2
2 3
2 4
```

Sample Output #1

```
FIRST
```

Explanation for the sample input/output #1

You can start your move by choosing (2, 4), whose effect was demonstrated in the following illustration.



The remaining black cells are (1, 3) and (1, 4), each of which will only toggle itself when chosen. Whichever your friend chooses on the next move, the you can choose the remaining black cell.

Sample Input #2

```
1
2 2
```

Sample Output #2

```
SECOND
```

Explanation for the sample input/output #2

You have only one cell to choose, and will toggle cells (1, 1), (1, 2), (2, 1), and (2, 2). Your friend and you will alternately choose the remaining black cells with your friend choosing the last black cell.

Sample Input #3

```
13
1 1
1 4
1 5
2 1
2 4
2 5
4 1
4 2
4 4
5 1
5 2
5 4
5 5
```

Sample Output #3

```
SECOND
```

Problem H

Twin Friends

You meet two new friends who are twins. The name of the elder twin is A , which consists of N characters. While the name of the younger twin is B , which consists of M characters. It is known that $N \leq M$.

You want to call each of them with a nickname. For the elder twin, you want to pick any permutation of A as the nickname. For the younger twin, you want to remove exactly $M - N$ characters from any permutation of B . Denote the nicknames of the elder twin and the younger twin as A' and B' , respectively.

You want the nicknames to satisfy the following requirement. For each i that satisfies $1 \leq i \leq N$, B'_i must be equal to either A'_i or the next letter that follows alphabetically after A'_i (if such a next letter exists).

Determine the number of different pairs of nicknames (A', B') that satisfy the requirement. Two pairs of nicknames are considered different if at least one of the nicknames are different. As the result might be large, find the answer modulo 998 244 353.

Input

The first line consists of two integers N M ($1 \leq N \leq M \leq 200\,000$).

The second line consists of a string A of length N .

The third line consists of a string B of length M .

All strings consist of only upper-case letters.

Output

Output a single integer representing number of different pairs (A', B') that satisfy the requirement, modulo 998 244 353.

Sample Input #1

```
3 4
AMA
ANAB
```

Sample Output #1

```
9
```

Explanation for the sample input/output #1

The 9 pairs are:

- (AAM, AAN),

- (AAM, ABN),
- (AAM, BAN),
- (AMA, ANA),
- (AMA, ANB),
- (AMA, BNA),
- (MAA, NAA),
- (MAA, NAB), and
- (MAA, NBA).

Sample Input #2

```
5 8
BINUS
BINANUSA
```

Sample Output #2

```
120
```

Explanation for the sample input/output #2

The 120 pairs are the pairs where A' is a permutation of BINUS and $B' = A'$.

Sample Input #3

```
15 30
BINUSUNIVERSITY
BINANUSANTARAUNIVERSITYJAKARTA
```

Sample Output #3

```
151362308
```

Sample Input #4

```
4 4
UDIN
ASEP
```

Sample Output #4

```
0
```

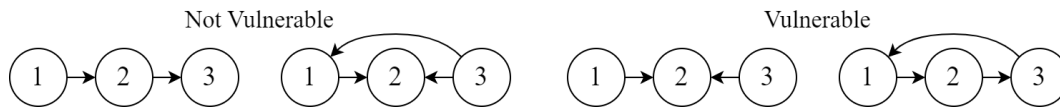

Problem I

Contingency Plan 2

You are working as a manager in The ICPC Company. In the company building, there are N computers (numbered from 1 to N). There are $N - 1$ cables, numbered from 1 to $N - 1$, that connect all the computers into a single network. Cable i connects computer U_i and V_i . Each cable can be set into emergency mode, where cable i only transfers data from computer U_i to computer V_i , but not the other way around. During a disaster, it is mandatory for all cables to be in emergency mode.

Through your research, you discover a new way to determine the vulnerability of a network. You want to add zero or more new cables to the current network such that it is **not vulnerable** during a disaster. Your network is not vulnerable if and only if there is **exactly one** permutation of 1 to N such that u appears before v in the permutation for all cables that connect computer u and v . In other words, it should have exactly one topological order.

The following illustration shows examples of not vulnerable networks and vulnerable networks.



For the not vulnerable networks, the only permutation that satisfies the requirement for the networks on the left and on the right are $[1, 2, 3]$ and $[3, 1, 2]$, respectively. Meanwhile, for the vulnerable networks, there are 2 permutations that satisfy the requirement for the network on the left: $[1, 2, 3]$ and $[3, 1, 2]$; while there is no permutation that satisfies the requirement for the network on the right.

You are interested in the minimum number of new cables that should be added to the current network such that it is not vulnerable during a disaster. Furthermore, you want to know, which pairs of computers should be connected by using the minimum number of cables. If there are several ways to connect, you can connect in any way of your choice. Under the given constraints, it can be proven that there exists a way to make the network not vulnerable.

Input

The first line consists of an integer N ($2 \leq N \leq 100\,000$).

Each of the next $N - 1$ lines consists of two integers $U_i V_i$ ($1 \leq U_i, V_i \leq N$). The input edges form a tree.

Output

The first line consists of an integer, representing the minimum number of new cables that should be added to the current network such that it is no longer vulnerable during a disaster. Denote this number as K and the new cables are numbered from 1 to K .

If K is not 0, then output K lines. Each of the next K lines consists of two integers $A_i B_i$, representing the computers that are connected by the new cable i . During a disaster, new cable i only transfers data from computer A_i to computer B_i , but not the other way around. If there exist several solutions, you can output any of them.

Sample Input #1

```
3
1 2
3 2
```

Sample Output #1

```
1
3 1
```

Sample Input #2

```
3
1 2
2 3
```

Sample Output #2

```
0
```

Sample Input #3

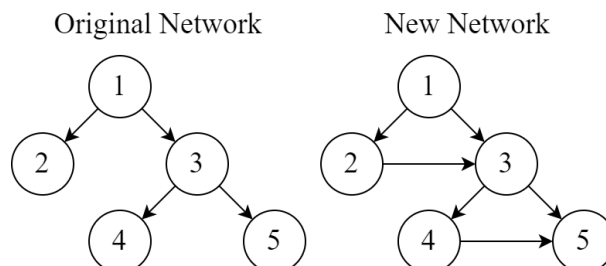
```
5
1 2
1 3
3 4
3 5
```

Sample Output #3

```
2
2 3
4 5
```

Explanation for the sample input/output #3

The following illustration shows the original network and the new network with the added cables during a disaster. The only permutation that satisfies the requirement is $[1, 2, 3, 4, 5]$.



Problem J

Count BFS Graph

You are currently researching a graph traversal algorithm called the Breadth First Search (BFS). Suppose you have an input graph of N nodes (numbered from 1 to N). The graph is represented by an adjacency matrix M , for which node u can traverse to node v if $M_{u,v}$ is 1, otherwise it is 0. Your algorithm will output the order the nodes are visited in the BFS. The pseudocode of the algorithm is presented as follows.

```
BFS( $M[1..N][1..N]$ ):  
    let A be an empty array  
    let Q be an empty queue  
  
    append 1 to A  
    push 1 to Q  
  
    while Q is not empty:  
        pop the front element of Q into u  
        for v = 1 to N:  
            if  $M[u][v] == 1$  and v is not in A:  
                append v to A  
                push v to Q  
  
    return A
```

During your research, you are interested in the following problem. Given an array A such that A is a permutation of 1 to N and $A_1 = 1$. How many **simple undirected graph** with N nodes and adjacency matrix M such that $\text{BFS}(M) = A$? Since the answer can be very large, calculate the answer modulo 998 244 353.

A simple graph has no self-loop ($M_{i,i} = 0$ for $1 \leq i \leq N$) and there is at most one edge that connects a pair of nodes. In an undirected graph, if node u is adjacent to node v , then node v is also adjacent to node u ; formally, $M_{u,v} = M_{v,u}$ for $1 \leq u < v \leq N$.

Two graphs are considered different if there is an edge that exists in one graph but not the other. In other words, two graphs are considered different if their adjacency matrices are different.

Input

The first line consists of an integer N ($2 \leq N \leq 5000$).

The second line consists of N integers A_i . The array A is a permutation of 1 to N and $A_1 = 1$.

Output

Output an integer representing the number of simple undirected graphs with N nodes and adjacency matrix M such that $\text{BFS}(M) = A$. Since the answer can be very large, output the answer modulo 998 244 353.

Sample Input #1

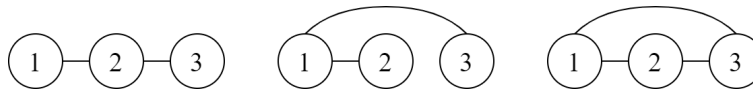
```
3
1 2 3
```

Sample Output #1

```
3
```

Explanation for the sample input/output #1

The following illustration shows all graphs that satisfy the requirements.

**Sample Input #2**

```
3
1 3 2
```

Sample Output #2

```
1
```

Explanation for the sample input/output #2

The only graph that satisfies the requirements is a graph with two edges: one that connects nodes 1 and 3, and another one that connects nodes 3 and 2.

Sample Input #3

```
5
1 3 2 4 5
```

Sample Output #3

```
17
```

Sample Input #4

```
11
1 2 3 4 5 6 7 8 9 10 11
```

Sample Output #4

```
379394847
```

Problem K

Deck-Building Game

You are playing a deck-building game with your friend. There are N cards, numbered from 1 to N . Card i has the value of A_i .

You want to build two decks; one for you and one for your friend. A card cannot be inside both decks, and it is allowed to not use all N cards. It is also allowed for a deck to be empty, i.e. does not contain any cards.

The **power** of a deck is represented as the bitwise XOR of the value of the cards in the deck. The power of an empty deck is 0.

The game is **balanced** if both decks have the same power.

Determine the number of ways to build two decks such that the game is balanced. Two ways are considered different if one of the decks contains at least one different card. Since the answer can be very large, calculate the answer modulo 998 244 353.

Input

The first line consists of an integer N ($2 \leq N \leq 100\,000$).

The following line consists of N integers A_i ($1 \leq A_i \leq 100\,000$).

Output

Output an integer representing the number of ways to build two decks such that the game is balanced. Output the answer modulo 998 244 353.

Sample Input #1

```
4
16 12 4 8
```

Sample Output #1

```
9
```

Explanation for the sample input/output #1

Denote S and T as the set of cards in your deck and your friend's deck, respectively. There are 9 ways to build the decks such that the game is balanced.

- $S = \{\}$ and $T = \{\}$. Both decks have the power of 0.
- $S = \{2, 3, 4\}$ and $T = \{\}$. Both decks have the power of 0.
- $S = \{\}$ and $T = \{2, 3, 4\}$. Both decks have the power of 0.
- $S = \{2, 4\}$ and $T = \{3\}$. Both decks have the power of 4.

- $S = \{3\}$ and $T = \{2, 4\}$. Both decks have the power of 4.
- $S = \{2, 3\}$ and $T = \{4\}$. Both decks have the power of 8.
- $S = \{4\}$ and $T = \{2, 3\}$. Both decks have the power of 8.
- $S = \{3, 4\}$ and $T = \{2\}$. Both decks have the power of 12.
- $S = \{2\}$ and $T = \{3, 4\}$. Both decks have the power of 12.

Sample Input #2

```
4
1 2 4 8
```

Sample Output #2

```
1
```

Explanation for the sample input/output #2

The only way to make the game balanced is to have both decks empty.

Sample Input #3

```
2
1 1
```

Sample Output #3

```
5
```

Explanation for the sample input/output #3

There are 5 ways to build the decks such that the game is balanced.

- $S = \{\}$ and $T = \{\}$. Both decks have the power of 0.
- $S = \{1, 2\}$ and $T = \{\}$. Both decks have the power of 0.
- $S = \{\}$ and $T = \{1, 2\}$. Both decks have the power of 0.
- $S = \{1\}$ and $T = \{2\}$. Both decks have the power of 1.
- $S = \{2\}$ and $T = \{1\}$. Both decks have the power of 1.

Sample Input #4

```
6
1 1 1 2 2 2
```

Sample Output #4

```
169
```

Problem L

Palindromic Parentheses

Construct a parentheses sequence consisting of N characters such that it is **balanced** and the length of its **longest palindromic subsequence (LPS)** is exactly K . Determine whether such a construction is possible. If there are several possible sequences, construct any of them.

A parentheses sequence consists of only character `(` and `)`. A parentheses sequence is **balanced** if each character `(` has a corresponding character `)` and the pairs of parentheses are properly nested. For example, `()`, `(( ))`, `(( ))()`, and `(( ( )) ( ))` are balanced. However, `)`, `((`, and `(( ))` are not balanced.

A sequence is **palindromic** if it reads the same backwards as forwards. For example, `((`, `)`, `(( ))`, and `(( ((` are palindromic. However, `()`, `)`, and `(( ))` are not palindromic.

A **subsequence** can be derived from another sequence by removing zero or more characters without changing the order of the remaining characters. For example, `(`, `))`, `(( ))`, and `(( ))()` are subsequence of `(( ))()`. However, `)((` and `(( ))` are not subsequence of `(( ))()`.

The **longest palindromic subsequence (LPS)** of a sequence is a subsequence with the maximum number of characters, derived from that sequence and it is palindromic. For example, the LPS of sequence `(( ))()` is `(( ))`, which can be obtained by removing the second and sixth characters. Therefore, the length of the LPS of `(( ))()` is 4.

Input

Input consists of two integers N K ($2 \leq N \leq 2000$; $1 \leq K \leq N$). N is an even number.

Output

If there is no such parentheses sequence such that it is balanced and the length of its LPS is exactly K , then output `-1`.

Otherwise, output a string of N characters, representing the parentheses sequence. If there are several possible answers, output any of them.

Sample Input #1

```
6 4
```

Sample Output #1

```
(( ))()
```

Sample Input #2

```
6 3
```

Sample Output #2

```
((()())
```

Explanation for the sample input/output #2

The LPS of `((()())` is either `((` by removing all `)` characters, or `))` by removing all `(` characters.

The output `((()())` also satisfies the requirements.

Sample Input #3

```
4 1
```

Sample Output #3

```
-1
```

Explanation for the sample input/output #3

The only possible balanced parentheses sequences are `((()))` and `()()`. The length of the LPS of `((()))` and `()()` are 2 and 3, respectively.

Sample Input #4

```
14 11
```

Sample Output #4

```
()((())()())()
```

Explanation for the sample input/output #4

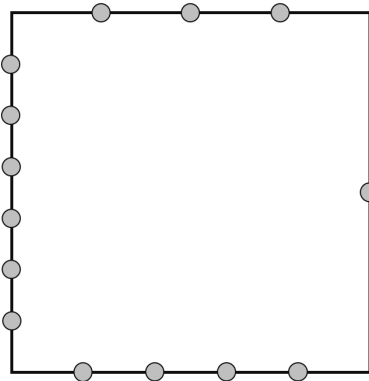
The LPS of `()((())()())()` is `)())()())()`, which can be obtained by removing the first, fourth, and fifth characters.

Problem M

Triangle Construction

You are given a regular N -sided polygon. Label one arbitrary side as side 1, then label the next sides in clockwise order as side 2, 3, $\dots$, N . There are A_i special points on side i . These points are positioned such that side i is divided into $A_i + 1$ segments with equal length.

For instance, suppose that you have a regular 4-sided polygon, i.e., a square. The following illustration shows how the special points are located within each side when $A = [3, 1, 4, 6]$. The uppermost side is labelled as side 1.



You want to create as many **non-degenerate triangles** as possible while satisfying the following requirements. Each triangle consists of 3 distinct special points (not necessarily from different sides) as its corners. Each special point can only become the corner of at most 1 triangle. All triangles must not intersect with each other.

Determine the maximum number of non-degenerate triangles that you can create.

A triangle is **non-degenerate** if it has a positive area.

Input

The first line consists of an integer N ($3 \leq N \leq 200\,000$).

The following line consists of N integers A_i ($1 \leq A_i \leq 2 \cdot 10^9$).

Output

Output a single integer representing the maximum number of non-degenerate triangles that you can create.

Sample Input #1

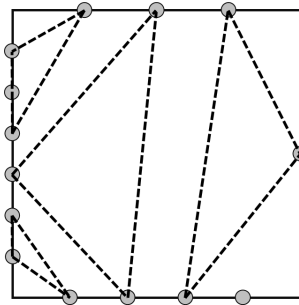
```
4
3 1 4 6
```

Sample Output #1

4

Explanation for the sample input/output #1

One possible construction which achieves maximum number of non-degenerate triangles can be seen in the following illustration.



Sample Input #2

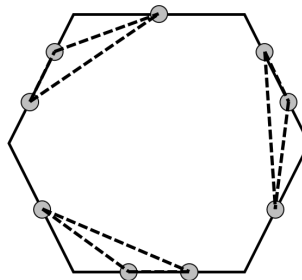
6
1 2 1 2 1 2

Sample Output #2

3

Explanation for the sample input/output #2

One possible construction which achieves maximum number of non-degenerate triangles can be seen in the following illustration.



Sample Input #3

3
1 1 1

Sample Output #3

1